

DESIGN AND ANALYSIS OF ALGORITHMS

35 Page Detailed Exam Notes

Unit I: Algorithm and Complexity Basics

Learning target: Understand what an algorithm is and how efficiency is measured.

These 35-page exam notes are based on the uploaded syllabus of Design and Analysis of Algorithms. An algorithm is a finite, ordered sequence of unambiguous steps that converts input into output and terminates after finite time. A good algorithm is correct, efficient, simple to implement, and scalable. Analysis of algorithm means estimating resources before actual execution. Main resources are time and memory. We usually analyze in terms of input size n , because exact machine time depends on hardware, compiler and programming language.

Important Points

Course Outcomes: complexity evaluation, Prim MST, 8-queen backtracking, TSP branch and bound, 0/1 knapsack DP.

Every algorithm answer should include: idea, steps/pseudocode, example/use, time complexity, space complexity, limitations.

Characteristics: input, output, definiteness, finiteness, effectiveness, correctness.

Time complexity counts elementary operations as a function of n .

Space complexity counts total memory used: input space + auxiliary space.

Worst case is used most often because it gives an upper guarantee.

Average case needs probability distribution of inputs; best case is usually optimistic.

Quick Table

Case	Meaning	Example
Best	Minimum operations	Linear search finds key at first position
Average	Expected operations	Key is equally likely at any position
Worst	Maximum operations	Linear search key absent or last position

Remember

Efficiency is compared by growth rate, not by exact seconds.

For large n , $n \log n$ is much better than n^2 .

Practice Questions

Define/explain Algorithm and Complexity Basics with a neat example.

Write algorithm steps or key method for Algorithm and Complexity Basics, then mention time and space complexity.

State two advantages, two limitations and one exam application of Algorithm and Complexity Basics.

Unit I: Time and Space Complexity

Learning target: Evaluate time-space requirements and understand trade-off.

Time complexity is commonly written using asymptotic notation. Space complexity includes fixed part such as program variables and variable part such as recursion stack, arrays, tables and dynamic structures. Time-space tradeoff means reducing time by using extra memory or reducing memory by accepting extra computation. Dynamic programming uses this tradeoff: it stores subproblem answers to avoid recomputation.

Important Points

Example: Binary search time $O(\log n)$, space $O(1)$ iterative, $O(\log n)$ recursive due to stack.

Merge sort time $O(n \log n)$, extra space $O(n)$ for merging.

Quick sort average $O(n \log n)$, worst $O(n^2)$, stack $O(\log n)$ average.

Hashing gives average $O(1)$ search using extra table memory.

Precomputation stores answers in advance to save query time.

Quick Table

Complexity	Growth	Typical Algorithm
$O(1)$	constant	array access
$O(\log n)$	slow	binary search
$O(n)$	linear	linear scan
$O(n \log n)$	near linear	merge sort, heap sort
$O(n^2)$	quadratic	simple nested loops
$O(2^n)$	exponential	subset generation

Practice Questions

Define/explain Time and Space Complexity with a neat example.

Write algorithm steps or key method for Time and Space Complexity, then mention time and space complexity.

State two advantages, two limitations and one exam application of Time and Space Complexity.

Unit I: Asymptotic Notations

Learning target: Write upper, lower and tight bounds correctly.

Asymptotic notation describes algorithm growth for large input size. It ignores constants and lower-order terms. Big-O gives upper bound, Omega gives lower bound, and Theta gives tight bound. These notations help compare algorithms independently of hardware. Example: $3n^2 + 5n + 10$ is $\Theta(n^2)$, because n^2 dominates for large n .

Important Points

Big-O: $f(n) = O(g(n))$ if f grows no faster than $c.g(n)$ after n_0 .
Omega: $f(n) = \Omega(g(n))$ if f grows at least $c.g(n)$ after n_0 .
Theta: $f(n) = \Theta(g(n))$ if both upper and lower bounds match.
Little-o means strictly smaller growth; little-omega means strictly larger growth.
Drop constants: $O(2n) = O(n)$; drop lower terms: $O(n^2+n) = O(n^2)$.

Quick Table

Expression	Simplified Bound	Reason
$5n + 20$	$\Theta(n)$	linear term dominates constant
$2n^2 + n \log n$	$\Theta(n^2)$	n^2 dominates $n \log n$
$\log n + 100$	$\Theta(\log n)$	constant ignored for large n
$n(n-1)/2$	$\Theta(n^2)$	quadratic pair count

Practice Questions

Define/explain Asymptotic Notations with a neat example.

Write algorithm steps or key method for Asymptotic Notations, then mention time and space complexity.

State two advantages, two limitations and one exam application of Asymptotic Notations.

Unit I: Recurrences and Solving Techniques

Learning target: Analyze recursive algorithms using recurrence relations.

A recurrence expresses running time of a recursive algorithm in terms of smaller input sizes. Divide-and-conquer algorithms naturally form recurrences. Three common solving methods are substitution, recursion tree, and Master Theorem. In exams, identify a , b and $f(n)$ in $T(n)=aT(n/b)+f(n)$.

Important Points

Substitution method: guess answer and prove by induction.

Recursion tree: expand levels, calculate work per level and sum all levels.

Master theorem: directly solves many divide and conquer recurrences.

Merge sort: $T(n)=2T(n/2)+n = \Theta(n \log n)$.

Binary search: $T(n)=T(n/2)+1 = \Theta(\log n)$.

Quick Table

Form	Condition	Answer
$T(n)=aT(n/b)+f(n)$	$f(n)$ smaller than $n^{\log_b a}$	$\Theta(n^{\log_b a})$
$T(n)=aT(n/b)+f(n)$	$f(n)$ equal to $n^{\log_b a}$	$\Theta(n^{\log_b a} \log n)$
$T(n)=aT(n/b)+f(n)$	$f(n)$ larger and regular	$\Theta(f(n))$

Remember

Always mention base case $T(1)=\Theta(1)$.

Master theorem is not suitable for all irregular recurrences.

Practice Questions

Define/explain Recurrences and Solving Techniques with a neat example.

Write algorithm steps or key method for Recurrences and Solving Techniques, then mention time and space complexity.

State two advantages, two limitations and one exam application of Recurrences and Solving Techniques.

Unit I: Divide and Conquer Technique

Learning target: Use divide, conquer and combine pattern.
Divide and conquer solves a large problem by dividing it into smaller independent subproblems, solving them recursively, and combining their results. It works best when subproblems are similar in form and balanced in size. The method improves performance by reducing repeated comparisons or by exploiting sorted/structured data.

Important Points

Divide: split input into two or more parts.
Conquer: solve each part recursively or directly if small.
Combine: merge partial solutions into final answer.
Examples: binary search, merge sort, quick sort, heap sort conceptually, Strassen multiplication.
Advantages: simple analysis, parallelism possible, often efficient.
Disadvantages: recursion overhead, extra space, poor performance if split is unbalanced.

Quick Table

Algorithm	Divide	Combine	Time
Binary Search	one half selected	none	$O(\log n)$
Merge Sort	two halves	merge sorted halves	$O(n \log n)$
Quick Sort	pivot partitions	none heavy	avg $O(n \log n)$
Strassen	matrix subblocks	matrix additions	$O(n^{2.807})$

Practice Questions

Define/explain Divide and Conquer Technique with a neat example.
Write algorithm steps or key method for Divide and Conquer Technique, then mention time and space complexity.
State two advantages, two limitations and one exam application of Divide and Conquer Technique.

Unit I: Binary Search

Learning target: Apply iterative and recursive binary search.
Binary search finds an element in a sorted array by repeatedly comparing with the middle element and discarding half of the search space. It is a classic divide-and-conquer algorithm. The array must be sorted. If data is unsorted, sorting cost must also be considered.

Important Points

Input: sorted array A[0..n-1], key x.
If $x == A[mid]$, answer found.
If $x < A[mid]$, search left half; otherwise search right half.
Iterative version uses constant extra space.
Recursive version is shorter but uses recursion stack.

Algorithm / Pseudocode

```
low = 0, high = n-1
while low <= high:
    mid = (low + high) // 2
    if A[mid] == x: return mid
    elif x < A[mid]: high = mid - 1
    else: low = mid + 1
return -1
```

Quick Table

Measure	Value
Best	$O(1)$
Worst	$O(\log n)$
Space iterative	$O(1)$
Space recursive	$O(\log n)$

Practice Questions

Define/explain Binary Search with a neat example.
Write algorithm steps or key method for Binary Search, then mention time and space complexity.
State two advantages, two limitations and one exam application of Binary Search.

Unit I: Merge Sort

Learning target: Evaluate merge sort algorithm and complexity.
Merge sort divides the array into two halves, recursively sorts both halves, and merges the sorted halves. It is stable and guarantees $O(n \log n)$ time in best, average and worst cases. Its main limitation is extra $O(n)$ auxiliary memory.

Important Points

Divide array until each subarray has one element.
Merge two sorted arrays using two pointers.
Stable: equal elements maintain original relative order.
Useful for linked lists and external sorting.
Recurrence: $T(n)=2T(n/2)+\Theta(n)$.

Algorithm / Pseudocode

```
mergeSort(A, l, r):  
    if l < r:  
        m = (l+r)//2  
        mergeSort(A, l, m)  
        mergeSort(A, m+1, r)  
        merge(A, l, m, r)
```

Quick Table

Property	Merge Sort
Best/Average/Worst	$O(n \log n)$
Extra Space	$O(n)$
Stable	Yes
In-place	No

Remember

For CO1, write recurrence and Master theorem result clearly.

Practice Questions

Define/explain Merge Sort with a neat example.
Write algorithm steps or key method for Merge Sort, then mention time and space complexity.
State two advantages, two limitations and one exam application of Merge Sort.

Unit I: Quick Sort

Learning target: Understand partitioning and average-case efficiency.
Quick sort chooses a pivot, partitions elements smaller than pivot to one side and larger elements to the other, then recursively sorts partitions. It is very fast in practice due to good cache behavior and in-place partitioning. Worst case occurs when pivot repeatedly gives highly unbalanced partitions, such as already sorted input with poor pivot choice.

Important Points

Partition is the key operation.
Average recurrence: $T(n)=2T(n/2)+\Theta(n)$.
Worst recurrence: $T(n)=T(n-1)+\Theta(n)$.
Randomized pivot reduces chance of worst case.
Not stable by default.

Algorithm / Pseudocode

```
quickSort(A, low, high):  
    if low < high:  
        p = partition(A, low, high)  
        quickSort(A, low, p-1)  
        quickSort(A, p+1, high)
```

Quick Table

Case	Time
Best	$O(n \log n)$
Average	$O(n \log n)$
Worst	$O(n^2)$
Auxiliary Space	$O(\log n)$ average stack

Practice Questions

Define/explain Quick Sort with a neat example.
Write algorithm steps or key method for Quick Sort, then mention time and space complexity.
State two advantages, two limitations and one exam application of Quick Sort.

Unit I: Heap Sort

Learning target: Use heap structure for in-place sorting.
Heap sort uses a binary heap, usually max-heap, to repeatedly place the largest element at the end of the array. A heap is a complete binary tree represented as an array where parent value is greater than or equal to its children for max-heap. Heap sort gives guaranteed $O(n \log n)$ time and $O(1)$ extra space, but it is not stable.

Important Points

- Build max heap from array in $O(n)$.
- Swap root with last element.
- Reduce heap size and heapify root.
- Repeat until heap size becomes 1.
- Parent index: $(i-1)//2$; children: $2i+1$ and $2i+2$.

Algorithm / Pseudocode

```
heapSort (A) :  
    buildMaxHeap (A)  
    for i = n-1 downto 1:  
        swap A[0], A[i]  
        heapSize = heapSize - 1  
        maxHeapify(A, 0)
```

Quick Table

Feature	Value
Time	$O(n \log n)$
Build heap	$O(n)$
Extra Space	$O(1)$
Stable	No

Practice Questions

- Define/explain Heap Sort with a neat example.
- Write algorithm steps or key method for Heap Sort, then mention time and space complexity.
- State two advantages, two limitations and one exam application of Heap Sort.

Unit I: Strassen Matrix Multiplication

Learning target: Know divide-and-conquer improvement over classical multiplication.
Classical matrix multiplication for two $n \times n$ matrices takes $O(n^3)$ time. Strassen's algorithm reduces multiplications from 8 to 7 for each block division, increasing additions but reducing asymptotic time. Matrices are split into four submatrices. The seven products P1 to P7 are combined to obtain the result blocks.

Important Points

Classical divide-and-conquer: 8 recursive multiplications of size $n/2$.
Strassen: 7 recursive multiplications plus additions/subtractions.
Recurrence: $T(n)=7T(n/2)+\Theta(n^2)$.
Time: $O(n^{\log_2 7})$ approximately $O(n^{2.807})$.
Practical only for large matrices due to overhead and numerical issues.

Quick Table

Method	Recursive Multiplications	Time
Classical	8	$O(n^3)$
Strassen	7	$O(n^{2.807})$

Remember

Exam point: fewer multiplications are more valuable than extra additions for large n .

Practice Questions

Define/explain Strassen Matrix Multiplication with a neat example.
Write algorithm steps or key method for Strassen Matrix Multiplication, then mention time and space complexity.
State two advantages, two limitations and one exam application of Strassen Matrix Multiplication.

Unit I: Code Tuning Techniques

Learning target: Improve program performance without changing algorithmic result.
Code tuning means small implementation-level changes that reduce execution time or memory overhead. It should be done after choosing a good algorithm, not as a replacement for algorithm design. Measure before and after tuning because premature optimization can make code complex without real benefit.

Important Points

Loop optimization: move invariant calculations outside loop, reduce nested loops, unroll small loops when useful.
Data transfer optimization: avoid unnecessary copying, pass large objects by reference, use cache-friendly arrays.
Logic optimization: simplify conditions, short-circuit frequent cases first, avoid repeated function calls.
Use appropriate data structures: hash table, heap, balanced tree, adjacency list.
Avoid recomputation using memoization or precomputed tables.

Quick Table

Technique	Example
Loop invariant movement	compute len once before loop
Strength reduction	replace expensive operation with cheaper equivalent
Cache locality	access arrays sequentially
Early exit	break when answer found

Practice Questions

Define/explain Code Tuning Techniques with a neat example.
Write algorithm steps or key method for Code Tuning Techniques, then mention time and space complexity.
State two advantages, two limitations and one exam application of Code Tuning Techniques.

Unit II: Greedy Strategy

Learning target: Understand greedy choice and optimal substructure.

A greedy algorithm builds the solution step by step by making the locally best choice at each stage. It never revisits previous choices. Greedy works only when local optimal choices lead to a globally optimal solution. Two important properties are greedy-choice property and optimal substructure.

Important Points

Greedy-choice property: a global optimum can be reached by choosing local optimum first.

Optimal substructure: optimal solution contains optimal solutions to subproblems.

Greedy is usually faster and simpler than DP.

Greedy may fail for 0/1 knapsack but works for fractional knapsack.

Correctness proof is essential: exchange argument or cut property.

Quick Table

Problem	Greedy Works?	Reason
Fractional knapsack	Yes	items divisible
0/1 knapsack	No	choice may block better combination
MST	Yes	cut property
Dijkstra	Yes with nonnegative weights	settled vertex distance final

Practice Questions

Define/explain Greedy Strategy with a neat example.

Write algorithm steps or key method for Greedy Strategy, then mention time and space complexity.

State two advantages, two limitations and one exam application of Greedy Strategy.

Unit II: Optimal Merge Pattern

Learning target: Apply greedy method to minimize merge cost.

Optimal merge pattern combines multiple sorted files with minimum total cost. At each step, merge the two smallest files first. This is greedy because small files should be used earlier to avoid repeatedly paying high merge cost. A min-heap is used to repeatedly extract two smallest sizes.

Important Points

- Insert all file sizes into min-heap.
- Remove two minimum sizes a and b.
- Merge cost = a+b; add it to total cost.
- Insert a+b back into heap.
- Repeat until one file remains.

Algorithm / Pseudocode

```
total = 0
while heap.size > 1:
    a = extractMin(heap)
    b = extractMin(heap)
    total += a + b
    insert(heap, a+b)
```

Quick Table

Files	Greedy Merge	Cost
5, 10, 20, 30	(5+10)=15	15
15, 20, 30	(15+20)=35	50
35, 30	(30+35)=65	115

Remember

Same idea appears in Huffman coding.

Practice Questions

- Define/explain Optimal Merge Pattern with a neat example.
- Write algorithm steps or key method for Optimal Merge Pattern, then mention time and space complexity.
- State two advantages, two limitations and one exam application of Optimal Merge Pattern.

Unit II: Huffman Coding

Learning target: Use greedy strategy for optimal prefix codes.

Huffman coding creates variable-length binary codes based on character frequencies. Frequent characters get shorter codes and rare characters get longer codes. The algorithm repeatedly combines the two least frequent nodes, forming a binary tree. It gives an optimal prefix-free code.

Important Points

Prefix code: no codeword is prefix of another codeword.

Use min-priority queue by frequency.

Left/right edges are labeled 0/1.

Final code for a character is path from root to leaf.

Applications: file compression, data transmission.

Algorithm / Pseudocode

```
for each character c: insert leaf(c, freq[c])
while queue has more than one node:
    x = extractMin()
    y = extractMin()
    z = new node with freq x.freq + y.freq
    z.left = x; z.right = y
    insert z
```

Quick Table

Property	Huffman Coding
Strategy	combine two minimum frequencies
Data Structure	min heap
Time	$O(n \log n)$
Code Type	prefix-free

Practice Questions

Define/explain Huffman Coding with a neat example.

Write algorithm steps or key method for Huffman Coding, then mention time and space complexity.

State two advantages, two limitations and one exam application of Huffman Coding.

Unit II: Minimum Spanning Tree

Learning target: Find MST using greedy approach.
A spanning tree of a connected undirected graph includes all vertices and has exactly $V-1$ edges. A minimum spanning tree has minimum total edge weight. MST algorithms use greedy choices based on safe edges. Two famous algorithms are Kruskal and Prim.

Important Points

- MST has no cycle.
- If graph has V vertices, MST has $V-1$ edges.
- Cut property: lightest edge crossing a cut is safe.
- Kruskal sorts edges and avoids cycles.
- Prim grows one tree from a starting vertex.

Quick Table

Algorithm	Greedy Choice	Data Structure	Good For
Kruskal	smallest edge globally	DSU/Union-Find	sparse graphs
Prim	smallest edge from tree to outside	priority queue	dense/connected graphs

Practice Questions

- Define/explain Minimum Spanning Tree with a neat example.
- Write algorithm steps or key method for Minimum Spanning Tree, then mention time and space complexity.
- State two advantages, two limitations and one exam application of Minimum Spanning Tree.

Unit II: Prim's Algorithm

Learning target: Use greedy strategy to construct MST from a start vertex.
Prim's algorithm starts with any vertex and repeatedly adds the minimum weight edge that connects the current tree to a vertex outside the tree. It maintains $key[v]$, the minimum edge weight by which vertex v can be added to the MST. Parent array stores final MST edges.

Important Points

Initialize $key[start]=0$ and all others infinity.
Pick unvisited vertex u with minimum key.
Mark u included in MST.
For every neighbor v of u , update $key[v]$ if $edge(u,v)$ is smaller.
After V picks, parent array gives MST.

Algorithm / Pseudocode

```
Prim(G) :
    key[start] = 0
    for count = 1 to V:
        u = vertex with minimum key not in MST
        add u to MST
        for each edge (u,v,w) :
            if v not in MST and w < key[v]:
                key[v] = w; parent[v] = u
```

Quick Table

Implementation	Time
Adjacency matrix	$O(V^2)$
Binary heap + list	$O(E \log V)$
Space	$O(V+E)$

Practice Questions

Define/explain Prim's Algorithm with a neat example.
Write algorithm steps or key method for Prim's Algorithm, then mention time and space complexity.
State two advantages, two limitations and one exam application of Prim's Algorithm.

Unit II: Kruskal, Job Sequencing and Fractional Knapsack

Learning target: Compare common greedy algorithms.
Kruskal's algorithm sorts all edges by weight and adds the next lightest edge if it does not create a cycle. Union-Find supports cycle checking.
Job sequencing with deadlines selects profit-making jobs before deadlines using slots. Fractional knapsack selects items by maximum profit/weight ratio and can take fractions.

Important Points

Kruskal: sort edges, add safe edges until V-1 edges selected.
Union-Find operations: find parent, union sets.
Job sequencing: sort jobs by profit descending; place each job in latest free slot before deadline.
Fractional knapsack: sort by value/weight ratio and fill capacity greedily.
0/1 knapsack cannot be solved greedily in general.

Quick Table

Problem	Greedy Key	Complexity
Kruskal MST	minimum edge avoiding cycle	$O(E \log E)$
Job sequencing	highest profit first	$O(n \log n + nD)$
Fractional knapsack	highest ratio first	$O(n \log n)$

Practice Questions

Define/explain Kruskal, Job Sequencing and Fractional Knapsack with a neat example.
Write algorithm steps or key method for Kruskal, Job Sequencing and Fractional Knapsack, then mention time and space complexity.
State two advantages, two limitations and one exam application of Kruskal, Job Sequencing and Fractional Knapsack.

Unit II: Single Source Shortest Path

Learning target: Understand Dijkstra's greedy algorithm.

Dijkstra's algorithm finds shortest paths from a single source to all vertices in a graph with nonnegative edge weights. It repeatedly selects the unvisited vertex with minimum tentative distance and relaxes its outgoing edges. Once a vertex is selected, its shortest distance becomes final.

Important Points

Initialize $\text{dist}[\text{source}] = 0$ and all others infinity.

Pick vertex u with minimum dist among unvisited vertices.

Relax each edge: if $\text{dist}[u] + w < \text{dist}[v]$, update $\text{dist}[v]$.

Works only for nonnegative edge weights.

For negative weights, Bellman-Ford is needed.

Algorithm / Pseudocode

```
while priority queue not empty:
    u = extractMin()
    for each edge (u,v,w):
        if dist[u] + w < dist[v]:
            dist[v] = dist[u] + w
            parent[v] = u
```

Quick Table

Implementation	Time
Matrix	$O(V^2)$
Binary heap	$O((V+E) \log V)$
Condition	No negative edges

Practice Questions

Define/explain Single Source Shortest Path with a neat example.

Write algorithm steps or key method for Single Source Shortest Path, then mention time and space complexity.

State two advantages, two limitations and one exam application of Single Source Shortest Path.

Unit II: Correctness Proof of Greedy Algorithms

Learning target: Write proof that greedy choice is safe.
A greedy algorithm is not accepted only because it looks logical. It needs a correctness proof. Most greedy proofs show that there exists an optimal solution containing the greedy choice. Then the remaining problem is smaller and can be solved similarly. Common proof styles are exchange argument, cut property and induction.

Important Points

Exchange argument: replace part of an optimal solution with greedy choice without worsening result.
Cut property: in MST, minimum edge crossing any cut is safe.
Induction: prove greedy is correct after each step.
Counterexample is used to show greedy does not work for some problems.
For exam answers, state property, prove safe choice, then show optimal substructure.

Quick Table

Algorithm	Correctness Idea
Prim/Kruskal	cut property
Huffman	two least frequent symbols are siblings at deepest level
Fractional knapsack	exchange lower ratio item with higher ratio item
Job sequencing	latest slot keeps earlier slots free

Practice Questions

Define/explain Correctness Proof of Greedy Algorithms with a neat example.
Write algorithm steps or key method for Correctness Proof of Greedy Algorithms, then mention time and space complexity.
State two advantages, two limitations and one exam application of Correctness Proof of Greedy Algorithms.

Unit III: Dynamic Programming Concept

Learning target: Solve problems with overlapping subproblems and optimal substructure.
Dynamic programming stores solutions of subproblems and reuses them. It is suitable when the same subproblems occur repeatedly and the optimal answer can be built from optimal subanswers. DP may be top-down with memoization or bottom-up with tabulation.

Important Points

Optimal substructure: optimal solution uses optimal subproblem solutions.
Overlapping subproblems: recursive calls repeat same states.
Memoization: recursive + cache.
Tabulation: iterative table filling.
DP often converts exponential recursion into polynomial time.

Quick Table

Approach	Meaning	Example
Top-down	solve recursively and store	memoized Fibonacci
Bottom-up	fill table from base cases	knapsack table
State	parameters defining subproblem	i, capacity
Transition	formula between states	include/exclude item

Practice Questions

Define/explain Dynamic Programming Concept with a neat example.
Write algorithm steps or key method for Dynamic Programming Concept, then mention time and space complexity.
State two advantages, two limitations and one exam application of Dynamic Programming Concept.

Unit III: 0/1 Knapsack using DP

Learning target: Solve knapsack problem using dynamic programming.

In 0/1 knapsack, each item can be either selected completely or rejected. Given weights, profits and capacity W, maximize profit without exceeding W. Greedy ratio does not always work because items are indivisible. DP state $dp[i][w]$ means maximum profit using first i items with capacity w.

Important Points

If $weight[i] > w$, item cannot be included.

Otherwise choose max of exclude and include item.

Transition: $dp[i][w] = \max(dp[i-1][w], profit[i] + dp[i-1][w-weight[i]])$.

Base: $dp[0][w]=0$ and $dp[i][0]=0$.

Answer: $dp[n][W]$.

Algorithm / Pseudocode

```
for i = 1 to n:
  for w = 0 to W:
    if wt[i] <= w:
      dp[i][w] = max(dp[i-1][w], val[i]+dp[i-1][w-wt[i]])
    else:
      dp[i][w] = dp[i-1][w]
```

Quick Table

Measure	Value
Time	$O(nW)$
Space	$O(nW)$, optimized $O(W)$
Strategy	DP not greedy
Type	pseudo-polynomial

Practice Questions

Define/explain 0/1 Knapsack using DP with a neat example.

Write algorithm steps or key method for 0/1 Knapsack using DP, then mention time and space complexity.

State two advantages, two limitations and one exam application of 0/1 Knapsack using DP.

Unit III: Multistage Graph and Reliability Design

Learning target: Apply DP to staged decision problems.

A multistage graph is a directed weighted graph whose vertices are divided into stages. Edges go from one stage to the next. The objective is usually to find the minimum cost path from source to sink. DP solves it backward from destination. Reliability design allocates resources among components to maximize system reliability under cost/weight constraints.

Important Points

- For multistage graph, $cost[v] = \min \text{ over edges}(v,u) \text{ of } c(v,u) + cost[u]$.
- Destination cost is zero.
- Decision at each stage depends on best future cost.
- Reliability design uses DP where state may be component index and budget.
- DP is useful when choices are sequential and constrained.

Quick Table

Problem	State	Transition
Multistage graph	vertex v	minimum edge cost plus next state
Reliability	component i, budget b	choose redundancy level
Resource allocation	stage, resource left	maximize benefit

Practice Questions

- Define/explain Multistage Graph and Reliability Design with a neat example.
- Write algorithm steps or key method for Multistage Graph and Reliability Design, then mention time and space complexity.
- State two advantages, two limitations and one exam application of Multistage Graph and Reliability Design.

Unit III: Floyd-Warshall Algorithm

Learning target: Find all-pairs shortest paths using dynamic programming.

Floyd-Warshall finds shortest paths between every pair of vertices in a weighted graph. It allows negative edges but not negative cycles. The idea is to gradually allow intermediate vertices. $dist[i][j]$ is improved by checking whether path $i \rightarrow k \rightarrow j$ is shorter than current $i \rightarrow j$.

Important Points

Initialize $dist[i][j]$ with edge weights, 0 for $i=j$, infinity if no edge.

For each intermediate vertex k , update all pairs i,j .

Transition: $dist[i][j] = \min(dist[i][j], dist[i][k] + dist[k][j])$.

Negative cycle exists if $dist[i][i]$ becomes negative.

Also used for transitive closure with boolean operations.

Algorithm / Pseudocode

```
for k = 1 to V:
  for i = 1 to V:
    for j = 1 to V:
      if dist[i][k] + dist[k][j] < dist[i][j]:
        dist[i][j] = dist[i][k] + dist[k][j]
```

Quick Table

Measure	Value
Time	$O(V^3)$
Space	$O(V^2)$
Negative edges	Allowed
Negative cycle	Not allowed

Practice Questions

Define/explain Floyd-Warshall Algorithm with a neat example.

Write algorithm steps or key method for Floyd-Warshall Algorithm, then mention time and space complexity.

State two advantages, two limitations and one exam application of Floyd-Warshall Algorithm.

Unit IV: Backtracking Concept

Learning target: Use state-space tree and pruning.
Backtracking is a systematic search technique that builds a solution step by step and abandons a partial solution as soon as it is known that it cannot lead to a valid answer. It is mainly used for constraint satisfaction problems. The search is represented as a state-space tree.

Important Points

- Choose: make one possible decision.
- Explore: recursively continue.
- Unchoose: undo decision and try next option.
- Pruning reduces unnecessary exploration.
- Examples: 8 queens, graph coloring, Hamiltonian cycle, subset sum.

Quick Table

Term	Meaning
State-space tree	all possible partial and complete solutions
Promising node	partial solution that may lead to answer
Pruning	discarding non-promising branches
Feasibility test	checking constraints before deeper recursion

Practice Questions

- Define/explain Backtracking Concept with a neat example.
- Write algorithm steps or key method for Backtracking Concept, then mention time and space complexity.
- State two advantages, two limitations and one exam application of Backtracking Concept.

Unit IV: Eight Queens Problem

Learning target: Apply backtracking to place queens safely.

The 8 queens problem asks to place 8 queens on an 8x8 chessboard so that no two queens attack each other. A queen attacks same row, same column and diagonals. Standard backtracking places one queen per row and tries columns. If a placement is safe, recursion moves to next row; otherwise it tries another column.

Important Points

Represent solution as `x[row] = column of queen in that row`.

No two queens can share same column.

Diagonal condition: `abs(row1-row2) != abs(col1-col2)`.

If row becomes 8, a full solution is found.

Backtracking explores only safe partial boards.

Algorithm / Pseudocode

```
solve(row):  
    if row == N: print solution  
    for col = 0 to N-1:  
        if isSafe(row, col):  
            place queen  
            solve(row + 1)  
            remove queen
```

Quick Table

Measure	Value
Naive possibilities	8^8
With one queen per row	8!
Technique	backtracking + pruning
General problem	N-Queens

Practice Questions

Define/explain Eight Queens Problem with a neat example.

Write algorithm steps or key method for Eight Queens Problem, then mention time and space complexity.

State two advantages, two limitations and one exam application of Eight Queens Problem.

Unit IV: Hamiltonian Cycle and Graph Coloring

Learning target: Use backtracking for graph problems.
A Hamiltonian cycle visits every vertex exactly once and returns to the starting vertex. Backtracking tries to build a path by adding adjacent unused vertices. Graph coloring assigns colors to vertices so that adjacent vertices have different colors. Backtracking tries colors vertex by vertex and rejects unsafe assignments.

Important Points

Hamiltonian cycle is different from Euler cycle: Hamiltonian focuses on vertices, Euler on edges.
Graph coloring decision problem asks whether graph can be colored with m colors.
Safe color check: no adjacent already-colored vertex has same color.
Both problems are computationally hard for general graphs.
Backtracking can solve small instances exactly.

Quick Table

Problem	Decision	Constraint
Hamiltonian cycle	next vertex in path	adjacent and not already used
Graph coloring	color for vertex	no same color on adjacent vertices
Subset sum	include/exclude item	sum target not violated

Practice Questions

Define/explain Hamiltonian Cycle and Graph Coloring with a neat example.
Write algorithm steps or key method for Hamiltonian Cycle and Graph Coloring, then mention time and space complexity.
State two advantages, two limitations and one exam application of Hamiltonian Cycle and Graph Coloring.

Unit IV: Branch and Bound Method

Learning target: Solve optimization problems using bounds.
Branch and bound is used for optimization problems. It explores a state-space tree like backtracking but uses bounds to avoid branches that cannot improve the current best solution. Branch means splitting into subproblems. Bound means estimating the best possible solution from a node.

Important Points

Maintain incumbent: best feasible solution found so far.
If node bound is worse than incumbent, prune it.
Live node: generated but not expanded.
E-node: node currently being expanded.
Least-cost branch and bound expands node with best bound first.

Quick Table

Backtracking	Branch and Bound
Mostly feasibility problems	Mostly optimization problems
Uses constraint pruning	Uses objective bound pruning
DFS common	BFS/priority queue common
Examples: N-queens	Examples: TSP, 0/1 knapsack

Practice Questions

Define/explain Branch and Bound Method with a neat example.
Write algorithm steps or key method for Branch and Bound Method, then mention time and space complexity.
State two advantages, two limitations and one exam application of Branch and Bound Method.

Unit IV: Traveling Salesman Problem using Branch and Bound

Learning target: Implement branch and bound idea for TSP.

In TSP, a salesman must visit each city exactly once and return to the starting city with minimum total cost. Branch and bound avoids checking all tours by computing lower bounds. A common matrix reduction method subtracts row minima and column minima; the sum gives a lower bound.

Important Points

- State represents partial tour.
- Branch by choosing next city or including/excluding an edge.
- Bound estimates minimum possible completion cost.
- If bound $\geq$ best tour cost, discard node.
- Priority queue can expand node with smallest lower bound.

Quick Table

Step	Action
1	Create cost matrix and set diagonal infinity
2	Reduce rows and columns to get initial lower bound
3	Branch to possible next cities
4	Add travel cost and new reduction cost
5	Update best complete tour and prune bad nodes

Remember

TSP brute force is $O(n!)$, so bounding is important.

Practice Questions

- Define/explain Traveling Salesman Problem using Branch and Bound with a neat example.
- Write algorithm steps or key method for Traveling Salesman Problem using Branch and Bound, then mention time and space complexity.
- State two advantages, two limitations and one exam application of Traveling Salesman Problem using Branch and Bound.

Unit IV: Lower Bound Theory

Learning target: Understand lower bounds and their use.
A lower bound is the minimum amount of work required by any algorithm to solve a problem under a model of computation. It helps prove that an algorithm is optimal. For example, comparison-based sorting has lower bound $\Omega(n \log n)$, so merge sort and heap sort are asymptotically optimal in that model.

Important Points

- Lower bound is about the problem, not only one algorithm.
- Upper bound is shown by giving an algorithm.
- If lower and upper bounds match, the algorithm is optimal.
- Decision tree model proves sorting lower bound.
- In branch and bound, lower bound estimates best possible solution under a node.

Quick Table

Problem	Known Lower Bound	Matching Algorithm
Comparison sorting	$\Omega(n \log n)$	merge sort, heap sort
Searching unsorted list	$\Omega(n)$	linear search
Searching sorted array	$\Omega(\log n)$	binary search
Matrix multiplication	not fully settled	advanced algorithms improve upper bound

Practice Questions

- Define/explain Lower Bound Theory with a neat example.
- Write algorithm steps or key method for Lower Bound Theory, then mention time and space complexity.
- State two advantages, two limitations and one exam application of Lower Bound Theory.

Unit V: Advanced Tree and Graph Algorithms

Learning target: Revise important graph and tree concepts.
Advanced graph algorithms build on traversal, shortest path, spanning tree and connectivity ideas. Graph representation affects complexity. Adjacency matrix is simple but takes $O(V^2)$ space. Adjacency list is efficient for sparse graphs. Tree algorithms often use traversal, balancing and divide-and-conquer.

Important Points

BFS explores level by level using queue; useful for unweighted shortest path.
DFS explores deeply using stack/recursion; useful for cycle detection and topological sort.
Topological sorting applies to DAGs.
Minimum spanning tree applies to undirected connected weighted graphs.
Shortest path algorithms depend on weight conditions.

Quick Table

Algorithm	Use	Time
BFS	unweighted shortest path	$O(V+E)$
DFS	components/cycle/topological	$O(V+E)$
Prim/Kruskal	MST	$O(E \log V)$ typical
Dijkstra	nonnegative SSSP	$O((V+E)\log V)$

Practice Questions

Define/explain Advanced Tree and Graph Algorithms with a neat example.
Write algorithm steps or key method for Advanced Tree and Graph Algorithms, then mention time and space complexity.
State two advantages, two limitations and one exam application of Advanced Tree and Graph Algorithms.

Unit V: NP-Hard and NP-Complete Problems

Learning target: Classify difficult computational problems.
P is the class of problems solvable in polynomial time. NP is the class of decision problems whose solution can be verified in polynomial time. A problem is NP-hard if every problem in NP can be reduced to it in polynomial time. A problem is NP-complete if it is both in NP and NP-hard.

Important Points

- P: efficiently solvable problems.
- NP: efficiently verifiable problems.
- NP-complete problems are the hardest problems inside NP.
- NP-hard problems may be optimization problems and may not be in NP.
- If any NP-complete problem has polynomial solution, then $P = NP$.

Quick Table

Class	Meaning	Example
P	polynomial-time solvable	MST, shortest path
NP	solution verifiable in polynomial time	Hamiltonian cycle
NP-complete	in NP and NP-hard	SAT, 3-SAT, Hamiltonian cycle
NP-hard	at least as hard as NP	TSP optimization

Practice Questions

- Define/explain NP-Hard and NP-Complete Problems with a neat example.
Write algorithm steps or key method for NP-Hard and NP-Complete Problems, then mention time and space complexity.
State two advantages, two limitations and one exam application of NP-Hard and NP-Complete Problems.

Unit V: Approximation Algorithms

Learning target: Use near-optimal solutions for hard problems.
For NP-hard optimization problems, exact algorithms may be too slow for large inputs. Approximation algorithms run in polynomial time and guarantee solution quality within a factor of optimum. Approximation ratio measures this guarantee.

Important Points

Used when exact optimal solution is computationally expensive.
Approximation ratio for minimization: $\text{algorithm cost} / \text{optimal cost}$.
For maximization: $\text{optimal value} / \text{algorithm value}$.
A 2-approximation for minimization gives solution at most twice optimum.
Heuristics may work well but do not always provide proof guarantee.

Quick Table

Problem	Approximation Idea
Vertex cover	pick both endpoints of uncovered edge
Metric TSP	MST-based tour gives bounded ratio
Set cover	greedy gives logarithmic approximation
Knapsack	FPTAS possible for 0/1 knapsack

Practice Questions

Define/explain Approximation Algorithms with a neat example.
Write algorithm steps or key method for Approximation Algorithms, then mention time and space complexity.
State two advantages, two limitations and one exam application of Approximation Algorithms.

Unit V: Randomized and Stream Algorithms

Learning target: Know algorithms for uncertainty and massive data.
Randomized algorithms use random choices to improve simplicity or expected performance. Example: randomized quick sort selects random pivot, making worst-case input unlikely. Stream algorithms process data arriving continuously and cannot store all input. They use small memory and often give approximate answers.

Important Points

Las Vegas algorithms always correct but running time is random.
Monte Carlo algorithms have bounded probability of error.
Randomized quick sort has expected $O(n \log n)$ time.
Stream model: one pass or few passes over data.
Examples: counting distinct elements, frequency estimation, heavy hitters.

Quick Table

Type	Feature	Example
Las Vegas	correct answer, random time	randomized quicksort variant
Monte Carlo	fast, small error chance	primality testing
Streaming	small memory	count-min sketch
Online	input arrives over time	online scheduling

Practice Questions

Define/explain Randomized and Stream Algorithms with a neat example.
Write algorithm steps or key method for Randomized and Stream Algorithms, then mention time and space complexity.
State two advantages, two limitations and one exam application of Randomized and Stream Algorithms.

Unit V: Parallel Algorithms

Learning target: Understand basic design and complexity of parallel algorithms.
Parallel algorithms divide computation among multiple processors to reduce running time. Important measures are work, span, speedup and efficiency. Work is total operations performed; span is longest dependency chain. A good parallel algorithm has enough independent tasks and low communication overhead.

Important Points

Speedup = serial time / parallel time.
Efficiency = speedup / number of processors.
Amdahl's law: serial portion limits maximum speedup.
Parallel divide-and-conquer works well when subproblems are independent.
Communication and synchronization overhead can reduce benefit.

Quick Table

Term	Meaning
Work	total computation over all processors
Span	critical path length
Speedup	how many times faster
Efficiency	processor utilization
Scalability	performance as processors increase

Practice Questions

Define/explain Parallel Algorithms with a neat example.
Write algorithm steps or key method for Parallel Algorithms, then mention time and space complexity.
State two advantages, two limitations and one exam application of Parallel Algorithms.

Final Revision: Must-Write Algorithms and Answers

Learning target: Last-page quick revision for exams.

Before the exam, revise definitions, properties, pseudocode and complexity table. In long answers, draw the state-space tree or table when possible. For numerical problems, show each step clearly because marks are given for method, not only final answer.

Important Points

Merge sort: recurrence $T(n)=2T(n/2)+n$, time $O(n \log n)$, space $O(n)$.

Prim: start vertex, key array, pick minimum key outside MST, update neighbors.

0/1 knapsack: include/exclude DP table, time $O(nW)$.

8 queens: one queen per row, check columns and diagonals.

TSP branch and bound: partial tour, lower bound, prune when bound is worse.

NP-complete: in NP + every NP problem reduces to it.

Quick Table

Question Type	Write These Points
Define/short note	definition + properties + example
Algorithm	idea + pseudocode + complexity
Compare	table of differences
Proof	property + argument + conclusion
Numerical	steps + table/tree + final answer

Remember

High scoring formulas: Master theorem, knapsack transition, Floyd transition, speedup and efficiency.

Practice Questions

Define/explain Must-Write Algorithms and Answers with a neat example.

Write algorithm steps or key method for Must-Write Algorithms and Answers, then mention time and space complexity.

State two advantages, two limitations and one exam application of Must-Write Algorithms and Answers.